

Role Task Breakdown — Credit Risk Production System

Task inventory for each engineering role in the Credit Risk Production Platform. ML predicts · Policy decides · LLM explains. Roles are ordered by position in the data → production pipeline: **Data Scientist** → **Data Engineer** → **ML Engineer** → **MLOps Engineer** → **LLMOps Engineer**

1. Data Scientist

Mission: Turn raw credit data into validated, trustworthy model performance evidence.

Exploratory Data Analysis (EDA)

- ☐ Profile the credit dataset (distributions, missing values, outliers, class balance)
- ☐ Identify leakage risks — especially features that could explain the suspiciously high metrics (99.5%+)
- ☐ Analyze correlation between features and the target (`credit_risk_flag`)
- ☐ Document data quality issues and domain assumptions

Feature Understanding

- ☐ Define the semantic meaning of each feature (income, utilization, delinquency history, etc.)
- ☐ Determine which features are legally/ethically usable for credit decisioning
- ☐ Propose feature transformations (log transforms, binning, ratios)

Experimentation

- ☐ Run baseline experiments for all six model families (GB, XGBoost, DT, RF, LR, KNN)
- ☐ Perform cross-validation and hyperparameter search
- ☐ Compare models with proper validation strategy (train/validation/test splits)
- ☐ Produce evaluation reports: Accuracy, Precision, Recall, F1, ROC AUC

Trustworthiness Validation

- ☐ Investigate whether the current 99.5%+ metrics are real or caused by data leakage
- ☐ Re-evaluate the best models on a untouched holdout set
- ☐ Check for duplicate rows between train and test sets
- ☐ Verify temporal consistency if data is time-ordered
- ☐ Write a validation memo: *"Are these results production-evidence or artifacts?"*

Deliverables

- ☐ EDA notebook/report
 - ☐ Model comparison report with honest metrics
 - ☐ Feature documentation (`model_features.json` source)
 - ☐ Leakage investigation findings
-

2. 🏗️ Data Engineer

Mission: Build reliable data pipelines that feed clean, correct, timely data to the ML system.

Data Pipeline

- ☐ Build ingestion pipeline from source systems to the modeling environment
- ☐ Implement data validation at entry (schema checks, type enforcement, range checks)
- ☐ Handle missing values and encoding consistently between training and inference
- ☐ Version datasets (raw → cleaned → feature-engineered)

Feature Store / Feature Consistency

- ☐ Ensure **training-serving skew is impossible**: the same preprocessing (`StandardScaler` , `LabelEncoder` , feature columns) must run identically in both paths
- ☐ Package preprocessing objects with the model bundle (`model_bundle.joblib`)
- ☐ Define the canonical input schema the API will validate against

Data Quality & Monitoring

- ☐ Set up data quality checks (freshness, completeness, drift detection on inputs)
- ☐ Log pipeline runs and data lineage (which data produced which model)
- ☐ Alert on pipeline failures or anomalous data volumes

Deliverables

- ☐ Reproducible data pipeline (scripts or orchestrated jobs)
 - ☐ Dataset versioning in place
 - ☐ Input schema definition shared with the API layer
-

3. 🧑 ML Engineer

Mission: Turn validated data science work into a reliable, packaged, servable model asset. (*Project scope: sections 22 & 25 of the workflow doc*)

Model Validation & Selection

- ☐ Validate the contents of `model_bundle.joblib` (6 models + preprocessing intact)
- ☐ Reproduce evaluation results independently of the data scientist
- ☐ Confirm the production candidate (Gradient Boosting) after the leakage investigation closes
- ☐ Benchmark inference latency per model (matters for selection, not just accuracy)

Packaging

- ☐ Structure the model bundle: models, scaler, encoders, feature columns, class labels
- ☐ Produce companion artifacts: `model_features.json`, `metadata.json`, `best_parameters.json`, feature importance
- ☐ Write a standardized scoring function: probabilities → `risk_score` → `risk_score_100`

MLflow Registration (Phase 1)

- ☐ Register all six models into MLflow with metrics and artifacts
- ☐ Tag Gradient Boosting version as `production-candidate` / `Staging`
- ☐ Define the promotion criteria a model must pass to reach `Production` stage

Serving Readiness

- ☐ Implement the prediction function used by FastAPI (single entry point: features → score)
- ☐ Write unit tests for the prediction path (schema, types, score range 0–100)
- ☐ Validate the model loads and serves correctly through the API before go-live

Deliverables

- ☐ Registered models in MLflow with complete metadata
 - ☐ Versioned model artifacts
 - ☐ Tested prediction service function
-

4. MLOps Engineer

Mission: Make the ML system reproducible, deployable, observable, versioned, and maintainable. (*Project scope: section 23*)

MLflow Lifecycle

- ☐ Set up MLflow tracking server and model registry
- ☐ Define model stage transitions: `None` → `Staging` → `Production` → `Archived`
- ☐ Wire the model-serving automation workflow to registry transitions

Orchestration

- ☐ Set up Kubeflow (or equivalent) for repeatable training/retraining pipelines
- ☐ Automate: data → train → evaluate → register as a single pipeline run

CI/CD (Phase 7)

- ☐ `ci.yaml` — code quality, tests, linting, Docker build validation on every push
- ☐ `models_serving.yaml` — validate model changes, promote approved versions in MLflow, connect to serving
- ☐ `cd.yaml` — build application image, deploy to Kubernetes after CI passes
- ☐ Enforce: **no model reaches production without passing validation gates**

Infrastructure (Phase 5)

- ☐ Dockerize FastAPI, MLflow, PostgreSQL, Prometheus, Grafana
- ☐ Deploy all services to Kubernetes with health checks and restart policies
- ☐ Configure scaling, service discovery, and secrets management (`MLFLOW_TRACKING_URI` , `DATABASE_URL` , etc.)

Observability (Phase 6)

- ☐ Prometheus metrics: prediction volume, model version in use, inference latency, prediction distribution
- ☐ Decision-route metrics: `AUTO_APPROVE` / `MANUAL_REVIEW` / `AUTO_REJECT` rates
- ☐ Grafana dashboards for engineering visibility
- ☐ Alerting: latency spikes, error rates, drift signals

Rollback Capability

- ☐ Prove rollback works: switch FastAPI from a bad `Production` version back to a validated one **without rebuilding the app**

Deliverables

- ☐ Running MLflow registry with staged models
 - ☐ Three working GitHub Actions workflows
 - ☐ Kubernetes deployment with health checks
 - ☐ Grafana dashboards + alert rules
-

5. 🧠 LLMOps Engineer

Mission: Make the LLM system versioned, observable, reproducible, policy-grounded, and safely integrated with the ML decision system. (*Project scope: section 24*)

RAG Foundation (Phase 4)

- ☐ Connect the existing ChromaDB store (`database/LLM/chroma_store/`)
- ☐ Verify knowledge base contents: Delinquency Classification, Fraud Typologies, Regulatory Risk Policy, Scorecard Cut-off Policy
- ☐ Version the RAG configuration (`rag_config.json`) and embedding model

LLM Integration

- ☐ Connect FastAPI to the Hugging Face LLM service through a controlled interface
- ☐ Enforce structured LLM output schema: `risk_level` , `reason[]` , `recommended_action` , `policy_references[]`
- ☐ Implement the rule: **LLM is invoked only for `MANUAL_REVIEW` cases** (or on explicit request), never as the score authority

Evaluation

- ☐ Build an evaluation set of review cases with expected explanations
- ☐ Measure: retrieval relevance (does RAG return the right policy?), response structure validity, faithfulness to retrieved context
- ☐ Test hallucination resistance: LLM must cite only retrieved policy references

LLM Observability

- ☐ Track per-interaction metadata: LLM provider/model, RAG version, prompt version, embedding model
- ☐ Metrics: request count, latency, token usage, error rate, fallback events
- ☐ Log retrieved policy references with each response for auditability

Safety & Guardrails

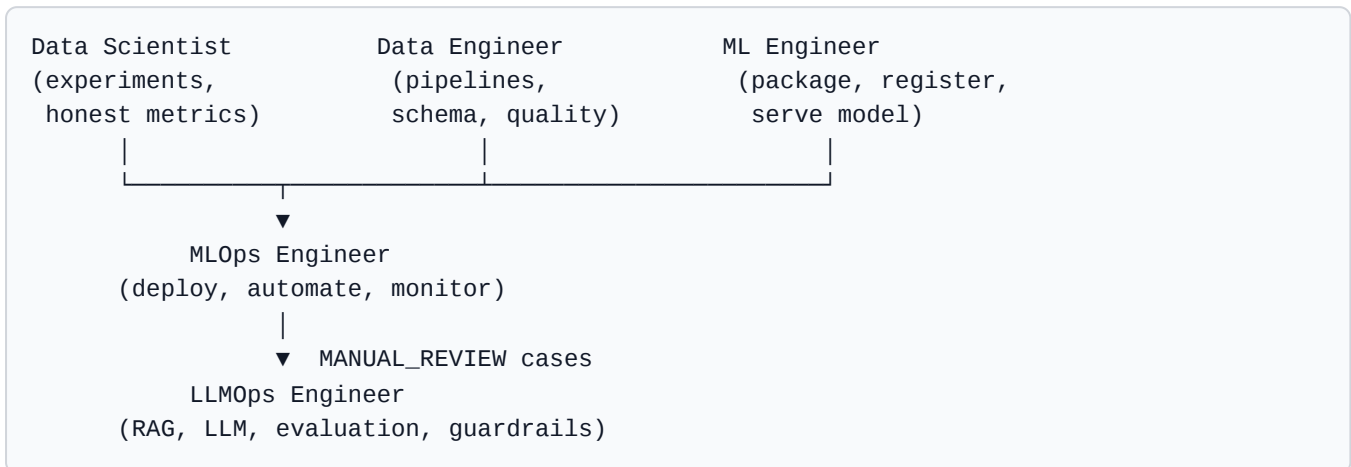
- ☐ Implement fallback behavior when the LLM is unavailable or times out (decision must still complete without explanation)
- ☐ Add input filtering and output validation
- ☐ Ensure the numerical ML score is never sourced from, or overridden by, an LLM response

Deliverables

- ☐ Working RAG + LLM pipeline integrated into the decision flow
- ☐ LLM evaluation report

- ❑ LLM monitoring dashboard + interaction metadata in PostgreSQL

How the Roles Connect



Handoff	From → To	Artifact
Validated dataset + findings	Data Scientist → Data Engineer / ML Engineer	EDA report, honest metrics
Clean, versioned data	Data Engineer → ML Engineer	Versioned datasets, schema
Registered model	ML Engineer → MLOps Engineer	MLflow <code>Staging</code> model
Deployed service	MLOps → everyone	Running API on Kubernetes
Review-case explanations	LLMOps → MLOps (monitoring)	LLM metrics, audit records

Cross-Cutting Responsibility: The Metrics Investigation

The suspiciously high evaluation results (99.5%+) are flagged in **every** role's tasks because each has a part to play:

- **Data Scientist** → prove or disprove leakage
- **Data Engineer** → rule out train/test contamination in the pipeline
- **ML Engineer** → reproduce results before registration
- **MLOps** → gate promotion on the investigation closing
- **LLMOps** → ensure no LLM explanation cites unvalidated metrics as fact

Traceability: Roles → Your Workflow Document (Ch. 1–26)

Every task in this file is grounded in your

`credit_risk_production_workflow_explanation.md`. Legend: ● = **Lead / primary owner** · ○ = **Supporting contributor** · blank = no direct impact

Role Impact Matrix

#	Chapter	 Data Sci	 Data Eng	 ML Eng	 MLOps	 LLMOps
1	 Production Goal	○	○	○	○	○
2	 Current ML Assets	○	●	●	○	○
3	 ML Model Responsibility	●	●	●		
4	 MLflow — Model Lifecycle	○		●	●	
5	 Production Model Selection	●	○	●	○	
6	 MLflow → FastAPI			●	●	
7	 FastAPI		○	●	●	○
8	 Decision Engine	○		○	○	●
9	 LLM Responsibility				○	●
10	 Existing RAG System				○	●
11	 LLMOps					●
12	 Hugging Face LLM Deployment				○	●
13	 PostgreSQL 15 Alpine		○	○	●	○
14	 Docker			○	●	○
15	 Kubernetes				●	

#	Chapter	 Data Sci	 Data Eng	 ML Eng	 MLOps	 LLMOps
16	 Prometheus & Grafana	○	○	○	●	●
17	 GitHub Actions			○	●	○
18	 Production Configuration			○	●	○
19	 Production Validation	●	●	●	●	●
20	 Complete Production Flow	○	○	○	○	○
21	 Tool Responsibilities					
22	 ML Engineer Scope			●		
23	 MLOps Engineer Scope				●	
24	 LLMOps Engineer Scope					●
25	 Production Development Order	○	○	●	●	●
26	 Final Target	○	○	○	○	○



Data Scientist — Where & How They Impact the Document

Chapter	Approach
Ch. 3 – ML Model Responsibility	Lead. Produces the "validated customer features → prediction → risk_score" logic. Defines the scoring policy example (<code>risk_score_100 = 90.73</code>) and validates that probabilities are calibrated.

Chapter	Approach
Ch. 5 – Production Model Selection	Lead. Owns the comparison table (GB 99.53% vs XGBoost 99.52% ...). Critically, drives the <i>"not accuracy only"</i> criteria — reliability, latency, probability quality. Flags the 99.5%+ metrics for the leakage investigation before they become "evidence."
Ch. 19 – Production Validation	Lead (model side). Defines what "evaluation metrics meet the required standard" means — sets the threshold and validation protocol.
Ch. 22 – ML Engineer Scope	Supporting. Hands the ML Engineer the validated findings so the bundle can be registered.
Ch. 1, 20, 26	Supporting. Informs the overall goal with honest feasibility ("these metrics may be too good to be true").

🔑 **Single biggest impact:** Ch. 5 — the trustworthiness of the entire platform depends on the Data Scientist proving the metrics are real.

🏗️ Data Engineer — Where & How They Impact the Document

Chapter	Approach
Ch. 2 – Current ML Assets	Lead (data side). Owns the pipeline that produced the dataset behind <code>model_bundle.joblib</code> ; documents data lineage so any model version can be traced back to its data.
Ch. 3 – ML Model Responsibility	Co-lead. Guarantees the sentence <i>"transformed using the same preprocessing logic used during model development"</i> is literally true — packages <code>StandardScaler</code> + <code>LabelEncoder</code> so training/serving skew is impossible.
Ch. 13 – PostgreSQL	Supporting. Defines which application records (predictions, decisions, audit) flow into PostgreSQL vs. what stays in MLflow.
Ch. 19 – Production Validation	Supporting. "Input schema is correct" is partly a data engineering deliverable — schema validation at ingestion.
Ch. 16 – Prometheus & Grafana	Supporting. Input data drift feeds the "prediction distribution" monitoring signal.

Chapter	Approach
Ch. 25 – Phase 1	Supporting. A model cannot be "validated" if its data pipeline is not reproducible.

🔑 **Single biggest impact:** Ch. 3 — the preprocessing-consistency guarantee is the #1 silent failure mode in production ML.

ML Engineer — Where & How They Impact the Document

Chapter	Approach
Ch. 2 – Current ML Assets	Co-lead. Validates <code>model_bundle.joblib</code> contents (6 models + preprocessing) as the Phase 1 starting point.
Ch. 4 – MLflow Model Lifecycle	Lead (model side). Executes the registration: <code>model_bundle.joblib</code> → MLflow → Metrics/Artifacts/Versions → Registry. Produces <code>model_features.json</code> , <code>metadata.json</code> , <code>best_parameters.json</code> .
Ch. 5 – Production Model Selection	Co-lead. Formally selects Gradient Boosting as production candidate; keeps other 5 registered for comparison.
Ch. 6 – MLflow → FastAPI	Lead. Builds the controlled relationship — the API loads the <i>approved version</i> from MLflow, not a local file. Enables rollback.
Ch. 7 – FastAPI	Co-lead. Implements <code>/api/v1/predict</code> and <code>/api/v1/risk/score</code> ; ensures the model receives validated features with correct types.
Ch. 19 – Production Validation	Lead (model side). Confirms predictions are valid, model serves through the API, metrics meet standard before exposure to traffic.
Ch. 22 – ML Engineer Scope	Lead. This chapter <i>is</i> this role's charter. Also owns the ⚠️ validation of unusually high results.
Ch. 25 – Phase 1 & 2	Lead. Phase 1 (ML Production) is entirely this role; Phase 2 (API) is shared with MLOps.

🔑 **Single biggest impact:** Ch. 4 + Ch. 6 — turning a notebook artifact into a versioned,

⚙️ MLOps Engineer — Where & How They Impact the Document

Chapter	Approach
Ch. 4 – MLflow	Co-lead (platform side). Runs the MLflow tracking server and registry as shared infrastructure; defines stage transitions.
Ch. 6 – MLflow → FastAPI	Co-lead. Operationalizes the promotion/rollback path.
Ch. 13 – PostgreSQL	Lead. Deploys and manages PostgreSQL 15 Alpine; enforces the MLflow ↔ PostgreSQL responsibility split (lifecycle vs. application records).
Ch. 14 – Docker	Lead. Packages all services into reproducible environments — <i>"the production application should not depend on a developer's local Python environment."</i>
Ch. 15 – Kubernetes	Lead. Deploys FastAPI, MLflow, PostgreSQL, Prometheus, Grafana as managed workloads with health checks, scaling, restart handling.
Ch. 16 – Prometheus & Grafana	Lead (platform). Builds the metrics pipeline: API, ML, and decision-route monitoring.
Ch. 17 – GitHub Actions	Lead. Owns all three workflows: <code>ci.yaml</code> , <code>models_serving.yaml</code> , <code>cd.yaml</code> — the gates that stop bad code/models from reaching production.
Ch. 18 – Production Configuration	Lead. Secrets via env vars / k8s secrets — <code>HUGGINGFACE_API_KEY</code> , <code>MLFLOW_TRACKING_URI</code> , <code>DATABASE_URL</code> , <code>POSTGRES_PASSWORD</code> never in Git.
Ch. 19 – Production Validation	Lead (deployment side). "Deployment should also be checked before the model is exposed to production traffic."
Ch. 23 – MLOps Scope	Lead. Charter: reproducible, deployable, observable, versioned, maintainable.
Ch. 25 – Phases 5, 6, 7	Lead. Infrastructure, Observability, CI/CD are entirely this role.

🔑 **Single biggest impact:** Ch. 17 — the GitHub Actions gates are what make everything else trustworthy; without them, validation is only a promise.

🧠 LLMOps Engineer — Where & How They Impact the Document

Chapter	Approach
Ch. 8 – Decision Engine	Lead (review branch). Owns what happens after <code>MANUAL_REVIEW</code> : triggers RAG + LLM, returns explanation into the decision flow.
Ch. 9 – LLM Responsibility	Lead. Enforces the boundary: LLM = intelligence/explanation layer, <i>never</i> the score authority.
Ch. 10 – Existing RAG System	Lead. Connects <code>chroma_store/</code> , curates the 4 knowledge bases (Delinquency, Fraud Typologies, Regulatory Policy, Scorecard Cut-off).
Ch. 11 – LLMOps	Lead. Tracks provider, model, RAG version, prompt version, embedding model, latency, tokens, errors, fallbacks, retrieved references.
Ch. 12 – Hugging Face Deployment	Lead. Builds the controlled interface + structured response schema (<code>risk_level</code> , <code>reason[]</code> , <code>recommended_action</code> , <code>policy_references[]</code>).
Ch. 16 – Prometheus & Grafana	Co-lead. Adds the LLM monitoring layer: requests, latency, errors, token usage, fallback count.
Ch. 19 – Production Validation	Lead (LLM side). Retrieval relevance, response structure, latency/error requirements.
Ch. 24 – LLMOps Scope	Lead. Charter: versioned, observable, reproducible, policy-grounded, safely integrated.
Ch. 25 – Phase 4	Lead. Entire LLMOps phase.

🔑 **Single biggest impact:** Ch. 9 + Ch. 12 — the guardrail that the ML score remains authoritative is the difference between an auditable credit system and an unaccountable one.

📌 Chapters With No Single Owner (Shared Context)

Chapter	Why it's shared
Ch. 1 – Production Goal	The three-area split (ML Eng / MLOps / LLMOps) concerns everyone; Data Sci + Data Eng ground it in data reality.
Ch. 20 – Complete Production Flow	The end-to-end diagram is the <i>integration contract</i> between all roles — each owns their box in the flow.

Chapter	Why it's shared
Ch. 21 – Tool Responsibilities	Reference table; each role reads their own row.
Ch. 25 – Development Order	Phases are sequential but cross-role: Phase 1 → ML Eng, Phase 4 → LLMOps, Phases 5–7 → MLOps, with Data Sci/Eng supporting throughout.
Ch. 26 – Final Target	The platform vision — every role's work converges here.